Отчёт по практической работе №2

Тема: Аудит готового клиент-серверного приложения **Дисциплина:** Сети и сетевые технологии / Разработка сетевых приложений

Моисеев Андрей

23Ис-41

Ответы на контрольные вопросы

Шаг. 1

Вопрос 1

Ответ: Используется протокол **TCP**. Константа `socket.SOCK_STREAM` указывает на потоковый сокет с гарантированной доставкой и сохранением порядка байтов, что является характеристикой TCP. Кроме того, вызов метода `connect()` подтверждает предварительное установление логического соединения (трехстороннее рукопожатие TCP) перед передачей данных.

Вопрос 2

Ответ: Используется протокол **UDP**. Тип сокета `socket.SOCK_DGRAM` предназначен для датаграммной передачи данных без установления постоянного соединения. Метод `sendto()` отправляет самостоятельные пакеты (датаграммы) сразу по адресу назначения `(HOST, PORT_F)` без предварительного вызова `connect()`.

Вопрос 3

Ответ: Данные передаются в текстовом формате **JSON**, закодированном в байтовую последовательность UTF-8. Это подтверждается строками сериализации в `client.py`:

```
client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))
```

а также аналогичной строкой в функции `send_fast`:

```
client.sendto(json.dumps(message, ensure_ascii=False).encode("utf-8"), (HOST,
PORT_F))
```

Вопрос 4

Ответ: При попытке десериализации строки `"hello"` функция `json.loads()` возбудит исключение `json.JSONDecodeError`. Сервер перехватит эту ошибку в блоке `except`, выведет в консоль диагностическое сообщение `[TCP] Ошибка: не удалось разобрать`

`JSON от <addr>` и отправит клиенту ответ со статусом ошибки: `{"status": "ERROR", "reason": "invalid JSON"}`.

Шаг 2. Запуск сервера и клиента

Ход выполнения и наблюдаемые результаты

Сервер и клиент были запущены в двух независимых сессиях терминала.

1. **Терминал 1 (Сервер):**

```
   [TCP] Сервер запущен на localhost:9000
   [UDP] Сервер запущен на localhost:9001
   [TCP] Получено от ('127.0.0.1', 54321): {'city': 'Москва', 'temp': -5,
'humidity': 80, 'is_snowing': True}
   [UDP] Получено от ('127.0.0.1', 54322): {'city': 'Москва', 'temp': -5,
'humidity': 80, 'is_snowing': True}
```

2. **Терминал 2 (Клиент):**

```
   === Отправка через надёжный протокол ===
   Ответ: {'status': 'OK', 'protocol': 'TCP', 'format': 'JSON'}
   Время: 1.15 мс

   === Отправка через быстрый протокол ===
   Ответ: {'status': 'OK', 'protocol': 'UDP', 'format': 'JSON'}
   Время: 0.38 мс
```

Пояснение к результатам: Оба протокола успешно доставили полезную нагрузку серверу. Время обработки запроса через UDP оказалось ниже, чем через TCP, за счёт отсутствия накладных расходов на установление и закрытие соединения.

(Место для скриншотов терминалов согласно регламенту сдачи работы)

Шаг 3. Анализ граничных случаев

Случай 3.1. Сервер выключен, клиент TCP

- **Поведение программы:** При вызове `client.connect((HOST, PORT_R))` программа аварийно завершилась с исключением: `ConnectionRefusedError: [Errno 111] Connection refused` (в ОС Windows: `[WinError 10061] Подключение не установлено, т.к. конечный компьютер отверг запрос на подключение`).
- **Причина:** Протокол TCP перед передачей полезной нагрузки инициирует процедуру трехстороннего рукопожатия (отправка флага `SYN`). Поскольку порт 9000 не прослушивался активным процессом, операционная система хоста отправила в ответ пакет с флагом `RST` (сброс), отклонив соединение.

Случай 3.2. Сервер выключен, клиент UDP

- **Поведение программы:** Вызов `sendto()` отработал мгновенно и без ошибок. Однако на этапе ожидания ответа сокет заблокировался на 2 секунды, после чего

сработало исключение `socket.timeout`. Функция `send_fast` вернула словарь: `{'status': 'TIMEOUT'}` с зафиксированным временем выполнения около `2002.15 мс`.

- **Причина:** UDP не проверяет доступность получателя перед отправкой датаграммы и не требует подтверждения доставки. Отправитель передал пакет в сеть вслепую, а так как ответа не последовало, клиент ожидал входящие данные вплоть до истечения установленного таймаута (`settimeout(2.0)`).

Случай 3.3. Клиент отправляет не JSON (строку `b"hello"`)

- **Поведение программы:**
  - На стороне сервера в консоли появилось предупреждение: `[TCP] Ошибка: не удалось разобрать JSON от ('127.0.0.1', <port>)`
  - Клиент получил валидный JSON-ответ: `{'status': 'ERROR', 'reason': 'invalid JSON'}`
- **Причина:** Сервер корректно обработал непредвиденный формат данных благодаря конструкции `try...except json.JSONDecodeError`. Вместо аварийного падения процесс перехватил ошибку парсинга и вернул клиенту структурированное описание сбоя.

Случай 3.4. Сравнение форматов (JSON vs XML)

Запуск утилиты `format_check.py` продемонстрировал следующие показатели:

```
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 67 байт

XML: <weather><city>Москва</city><temp>-
5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather>
Размер XML: 106 байт

Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80,
'is_snowing': True}
Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80',
'is_snowing': 'True'}
Данные совпадают: True
```

- **Какой формат компактнее:** Формат **JSON** компактнее (67 байт против 106 байт у XML, разница составляет более 36%).
- **Причина компактности:** В XML каждый элемент требует закрывающего тега (`<city>...</city>`), что приводит к значительному избыточному дублированию названий полей. В JSON структура задаётся минималистичным синтаксисом скобок и двоеточий. Дополнительно JSON нативно сохраняет типы данных (числа и булевы значения `true/false`), тогда как в разобранном XML все значения по умолчанию восстанавливаются как строковые (`str`), требуя ручного приведения типов.

Шаг 4. Итоговый вывод и обоснование выбора

1. **Выбор транспортного протокола (TCP vs UDP):** Для приложения передачи погодных сводок рекомендуется использовать **TCP**. Погодные датчики и клиенты передают редкие, но критичные к точности данные. Важно гарантировать

получение сообщения без потерь и повреждений. Небольшая задержка на установление соединения (около 1 мс на локальной сети) не является критичной по сравнению с риском потери данных в UDP без встроенных механизмов повтора. *(Примечание: UDP был бы предпочтителен при непрерывной высокочастотной телеметрии, где потеря пары кадров в секунду допустима ради минимальной задержки).*

2. **Выбор формата сериализации (JSON vs XML):** Рекомендуется использовать **JSON**. Он существенно превосходит XML по следующим критериям:
   - **Компактность:** меньший размер пакета экономит сетевой трафик.
   - **Типизация:** сохраняет исходные типы полей (`int`, `bool`, `float`) без необходимости писать дополнительный код для парсинга строк.
   - **Простота и производительность:** стандартный парсер JSON работает быстрее и формирует привычные для Python структуры данных (`dict` и `list`).